# LIBRARY STUDIO: Library Management System

A PROJECT REPORT

*Submitted by*

| ROLL NUMBER | REGISTRATION NO. | STUDENT CODE | STUDENT NAME |
|---|---|---|---|
| | | **BWU/MCA/26/015** | **Esita Hatua** |

*in partial fulfillment for the award of the degree*

*of*

**MASTERS OF COMPUTER APPLICATIONS**

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



# ABSTRACT

The Library Studio: Library Management System is a web-based application developed to simplify and manage the daily operations of a library. It provides an organized platform for maintaining book records, managing student or member information, and handling book issuing and returning activities.

The application is developed using Python and Flask for backend processing, HTML and CSS for the user interface, and JavaScript for client-side validation and interactive operations. The system uses a JSON file for storing and managing application data.

The main features include adding and managing books, registering members, issuing books, returning books, and calculating late-return fines. A book can be deleted from the catalogue only when it is available, while a member can be deleted only when no book is currently issued to that member.

The system also provides a dashboard displaying important library statistics, including total books, available books, issued books, and registered members. Graphical representations help users understand the current status of library resources.

The primary aim of this project is to reduce manual record-keeping, improve data organization, and make routine library operations more efficient and user-friendly.

**Keywords:** Library Management System, Python, Flask, JSON, JavaScript, Book Management, Member Management, Fine Calculation.

# TABLE OF CONTENTS


| Sl. No. | Contents | Page No. |
|---|---|---|
| 1 | Abstract | i |
| 2 | Table of Contents | ii – iii |
| 3 | List of Tables | v |
| 4 | List of Figures | vi |
| 5 | List of Symbols, Abbreviations and Nomenclature | vii |
| | **CHAPTER 1 – INTRODUCTION** | **1** |
| 1.1 | Introduction | 1 |
| 1.2 | Background of the Project | 1 |
| 1.3 | Overview of the Existing System | 1 |
| 1.4 | Proposed System | 1-2 |
| | **CHAPTER 2 – OBJECTIVE** | **3** |
| 2.1 | Primary Objective | 3 |
| 2.2 | Secondary Objectives | 3 |
| 2.3 | Functional Objectives | 3-4 |
| 2.4 | Technical Objectives | 4 |
| 2.5 | User-Centric Objectives | 4 |
| 2.6 | Administrative Objectives | 4 |
| 2.7 | Long-Term Objectives | 4 |
| | **CHAPTER 3 – PLANNING** | **5** |
| 3.1 | Introduction | 5 |
| 3.2 | Project Development Approach | 5 |
| 3.3 | Work Breakdown Structure (WBS) | 5-6 |
| 3.4 | Project Scheduling | 6 |
| 3.5 | Module Identification | 6 |
| 3.6 | Summary | 6 |
| | **CHAPTER 4 – REQUIREMENT ANALYSIS** | **7** |
| 4.1 | Introduction | 7 |
| 4.2 | Functional Requirements | 7 |
| 4.3 | Non-Functional Requirements | 8 |

| | | |
|---|---|---|
| 4.4 | Hardware and Software Requirements | 8 |
| 4.5 | Data Requirements | 8-9 |
| 4.6 | Summary | 9 |
| | **CHAPTER 5 – SYSTEM FLOW** | **10** |
| 5.1 | Context Level Diagram (CLD) | 10 |
| 5.2 | Data Flow Diagram – Level 0 | 10 |
| 5.3 | Use-Case Diagram | 11 |
| 5.4 | Entity-Relationship Diagram | 11 |
| 5.5 | Activity Diagram | 12 |
| 5.6 | Sequence Diagram | 12 |
| 5.7 | Book Issue and Return Flowchart | 12 |
| 5.8 | Summary | 12 |
| | **CHAPTER 6 – PROPOSED DESIGN** | **13** |
| 6.1 | Design Approach | 13 |
| 6.2 | System Architecture Design | 13 |
| 6.3 | User Interface Design | 14 |
| 6.4 | Data Storage Design | 14 |
| 6.5 | Design Consideration | 14 |
| 6.6 | Implementation Details | 14 |
| 6.7 | Summary | 14 |
| | **CHAPTER 7 – EXPERIMENTAL RESULT** | **15** |
| 7.1 | Overview of Experimental Setup | 15 |
| 7.2 | Functional Testing | 15 - 16 |
| 7.3 | Fine Calculation Results | 16 - 17 |
| 7.4 | Dashboard Analytics and Results | 17 - 18 |
| 7.5 | Performance Observations | 18 |
| 7.6 | Summary | 18 |
| | **CHAPTER 8 – LIMITATIONS & FUTURE SCOPE** | 19 |
| | **CHAPTER 9 – CONCLUSION** | **20** |
| | **REFERENCES / BIBLIOGRAPHY** | 21 |

# LIST OF TABLES


| Table No. | Table Title | Page No. |
|---|---|---|
| Table 3.1 | Project Development Schedule | 6 |
| Table 4.1 | Functional Requirements | 7 |
| Table 4.2 | Non-Functional Requirements | 8 |
| Table 4.3 | Hardware and Software Requirements | 8 |
| Table 6.1 | JSON Data Storage Structure | 14 |
| Table 7.1 | Consolidated System Testing | 15 |
| Table 7.2 | Fine Calculation Examples | 17 |

# LIST OF FIGURES


| Figure No. | Figure Title | Page No. |
|---|---|---|
| Figure 5.1 | Data Flow Diagram – Level 0 | 10 |
| Figure 5.2 | Use Case Diagram | 11 |
| Figure 5.3 | Entity Relationship Diagram (ERD) | 11 |
| Figure 5.4 | Activity Diagram | 12 |
| Figure 6.1 | System Architecture Diagram | 13 |

# LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE

| Abbreviation | Meaning |
|---|---|
| LMS | Library Management System |
| UI | User Interface |
| UX | User Experience |
| HTML | HyperText Markup Language |
| CSS | Cascading Style Sheets |
| JS | JavaScript |
| API | Application Programming Interface |
| JSON | JavaScript Object Notation |
| DFD | Data Flow Diagram |
| CLD | Context Level Diagram |
| ERD | Entity–Relationship Diagram |
| CRUD | Create, Read, Update and Delete |
| WBS | Work Breakdown Structure |
| HTTP | HyperText Transfer Protocol |
| IDE | Integrated Development Environment |
| AJAX | Asynchronous JavaScript and XML |
| MVC | Model–View–Controller |

CHAPTER 1

# INTRODUCTION

### 1.1 Introduction

A Library Management System is a software application designed to manage library activities such as maintaining book records, registering members, issuing books, and processing returned books. Traditional library management methods often involve manual record-keeping, which can require considerable time and effort.

The project titled "Library Studio: Library Management System" is a web-based application developed to simplify these activities. It provides a centralized interface through which library records and circulation activities can be managed efficiently.

The application is developed using Python and Flask for backend processing, HTML and CSS for the user interface, JavaScript for interactive operations, and JSON-based file storage for maintaining application data.

### 1.2 Background of the Project

Libraries manage different types of information, including book details, member records, and book circulation activities. Maintaining these records manually can make it difficult to track book availability, identify issued books, and calculate late-return fines.

The need for a simple computerized system motivated the development of Library Studio. The project focuses on organizing library information and reducing repetitive manual work through a web-based interface.

The system provides separate facilities for book management, member registration, book issuing, and book returning. It also displays important library statistics through a dashboard.

### 1.3 Overview of the Existing System

In a traditional manual library management system, book details, member information, and issue-return records may be maintained using registers or spreadsheets.

Some common limitations of such methods include:

- Maintaining records manually requires time and effort.
- Searching for book and member information can be difficult.
- Tracking issued and available books may require repeated checking.
- Calculating late-return fines manually can lead to errors.
- Preparing library statistics requires additional work.

These limitations indicate the need for a computerized system that organizes routine library operations.

### 1.4 Proposed System

The proposed Library Studio application provides a web-based solution for managing essential library activities.

The system includes the following features:

1. **Book Management:** Add and manage book records and maintain catalogue information.
2. **Member Management:** Register members and maintain their information.
3. **Book Issuing:** Issue available books to registered members and calculate a due date

of 14 days from the issue date.

4. **Book Returning:** Process returned books and update their availability.
5. **Fine Calculation:** Calculate a late-return fine of ₹5 per day after the 14-day borrowing period.
6. **Deletion Restrictions:** Allow book deletion only when the book is available and member deletion only when the member has no currently issued book.
7. **Dashboard and Analytics:** Display total books, available books, issued books, and registered members through dashboard statistics and charts.
8. **Input Validation:** Validate user inputs to reduce incorrect data entry.

The application uses JSON file-based storage to maintain its records. It also uses asynchronous operations for selected activities so that information can be updated without a complete page reload.

The proposed system provides a simple and organized method of handling common library operations. It aims to improve record management, reduce manual effort, and make library activities easier to perform.

CHAPTER 2

# OBJECTIVE

### 2.1 Primary Objective

The primary objective of the Library Studio: Library Management System is to develop a simple, efficient, and user-friendly web-based application for managing the basic activities of a library.

The system aims to reduce manual record-keeping, simplify book circulation, maintain member information, and provide accurate information about book availability. It also helps manage book returns and calculate fines for late returns.

### 2.2 Secondary Objectives

The secondary objectives of the project are:

- To maintain organized records of books available in the library.
- To register and manage library members.
- To simplify the process of issuing and returning books.
- To maintain accurate book availability information.
- To reduce errors caused by manual data entry.
- To provide a clear dashboard for monitoring library activities.
- To maintain library records using JSON-based storage.

### 2.3 Functional Objectives

The functional objectives describe the main operations that the proposed system is designed to perform.

- **Book Management:** Add new books, view book records, and manage the book catalogue.
- **Member Management:** Register new members and maintain their information.
- **Book Issuing:** Issue available books to registered members and assign a due date of 14 days from the issue date.
- **Book Returning:** Process returned books and update their availability status.
- **Fine Calculation:** Calculate a fine of ₹5 per day when a book is returned after the permitted 14-day borrowing period.

- **Deletion Control:** Prevent deletion of books that are currently issued and members who have currently issued books.
- **Dashboard Management:** Display total books, available books, issued books, and registered members.
- **Input Validation:** Validate user inputs and display appropriate error messages for invalid entries.

### 2.4 Technical Objectives

The technical objectives focus on the technologies and implementation methods used in the development of the application.

- To develop the backend using Python and the Flask framework.
- To create the user interface using HTML and CSS.
- To use JavaScript for client-side validation and interactive operations.
- To store application data using JSON file-based storage.
- To use asynchronous requests for selected form operations.
- To display library statistics using graphical representations through Chart.js.

### 2.5 User-Centric Objectives

The system aims to provide a simple and understandable interface for library operators. It reduces the effort required to search for information, manage records, and perform book circulation activities.

Input validation and appropriate feedback help users enter correct information and understand the result of each operation.

### 2.6 Administrative Objectives

The system maintains book and member information in an organized manner. It also applies restrictions to prevent the deletion of books and members involved in active circulation records.

The dashboard provides a summarized view of library resources, helping the operator monitor the current status of the library.

### 2.7 Long-Term Objectives

The project provides a foundation for future development. The system may be extended with database integration, user authentication, automated notifications, advanced reporting, and backup facilities to improve its reliability and scalability.

CHAPTER 3

# PLANNING

### 3.1 Introduction

Project planning is an important phase of software development. It helps to identify the activities, resources, and development stages required to complete a project successfully.

The planning of Library Studio focuses on requirement analysis, system design, application development, testing, and documentation. The project is divided into smaller tasks to ensure systematic development.

### 3.2 Project Development Approach

The Library Studio project follows a structured development approach. The development process is divided into the following phases:

1. **Project Initiation:** Selection of the project topic and identification of its main objectives.
2. **Requirement Analysis:** Identification of the functional and non-functional requirements.
3. **System Design:** Planning the application architecture, user interface, navigation, and data storage structure.
4. **Development:** Implementation of the backend using Python and Flask and the frontend using HTML, CSS, and JavaScript.
5. **Testing:** Verification of book management, member management, issuing, returning, and fine calculation.
6. **Documentation:** Preparation of diagrams, screenshots, tables, and the final project report.

### 3.3 Work Breakdown Structure (WBS)

The Work Breakdown Structure divides the project into smaller and manageable activities.

The major activities of Library Studio are:

1. **Project Initiation**
    - Selection of project topic.
    - Definition of project objectives.
    - Identification of basic requirements.
2. **Requirement Analysis**
    - Identification of book and member management requirements.
    - Analysis of book issuing and returning operations.
    - Identification of fine calculation rules.
3. **System Design**
    - Planning application architecture.
    - Designing the user interface.
    - Planning JSON-based data storage.
4. **Development**
    - Development of the Flask backend.
    - Development of the frontend interface.
    - Implementation of library management modules.
    - Integration of dashboard charts.
5. **Testing and Documentation**
    - Testing major application functions.
    - Preparing system diagrams.

- Preparing the final project report.

### 3.4 Project Scheduling

Project scheduling defines the sequence and planned duration of different development activities.

**Table 3.1: Project Development Schedule**

| Sl. No. | Project Activity | Planned Duration |
|---|---|---|
| 1 | Project Topic Selection and Planning | 1 Week |
| 2 | Requirement Analysis | 1 Week |
| 3 | System Design and Interface Planning | 1 Week |
| 4 | Backend and Data Storage Development | 2 Weeks |
| 5 | Frontend and Module Development | 2 Weeks |
| 6 | Integration and Testing | 1 Week |
| 7 | Documentation and Final Preparation | 1 Week |

### 3.5 Module Identification

The Library Studio application consists of the following major modules:

1. **Book Management Module:** Allows the operator to add, view, and delete books when they are not currently issued.
2. **Member Management Module:** Manages member registration and records.
3. **Book Issuing Module:** Issues available books to registered members and assigns a due date 14 days after the issue date.
4. **Book Returning Module:** Processes returned books and updates their availability.
5. **Fine Calculation Module:** Calculates a fine of ₹5 per day for books returned after the permitted borrowing period.
6. **Dashboard and Analytics Module:** Displays library statistics through summary information and charts.
7. **Input Validation Module:** Checks user inputs and helps prevent invalid information from being accepted.

### 3.6 Summary

The project planning phase provides a clear development structure for Library Studio. Dividing the project into different phases and modules helps organize the development process and supports systematic implementation and testing.

CHAPTER 4

# REQUIREMENT ANALYSIS

### 4.1 Introduction

Requirement analysis identifies the functions and resources required for the Library Studio application. It defines the system's functional and non-functional requirements.

### 4.2 Functional Requirements

Functional requirements describe the operations performed by the application.

The major functions include:

1. Book Management – Add, view, and manage book records.
2. Member Management – Register and manage library members.
3. Book Issuing – Issue available books to registered members.
4. Book Returning – Update book availability after return.
5. Fine Calculation – Calculate ₹5 per day for overdue books after 14 days.
6. Dashboard – Display library statistics and charts.
7. Input Validation – Validate user-entered information.
8. Data Storage – Maintain records in JSON format.

**Table 4.1: Functional Requirements**

| **Requirement** | **Description** |
|---|---|
| Book Management | Add and manage books. |
| Member Management | Register and manage members. |
| Issue/Return | Manage book circulation. |
| Fine Calculation | Calculate overdue fines. |
| Dashboard | Display library statistics. |
| Data Validation | Prevent invalid inputs. |
| Data Storage | Store records in JSON format. |

### 4.3 Non-Functional Requirements

Non-functional requirements describe the quality of the application.

The major requirements are usability, reliability, performance, maintainability, data integrity, and browser compatibility.

**Table 4.2: Non-Functional Requirements**

| Requirement | Description |
|---|---|
| Usability | Simple and user-friendly interface. |
| Reliability | Maintain consistent records. |
| Performance | Process routine operations efficiently. |
| Maintainability | Easy to modify and update. |
| Data Integrity | Maintain correct library records. |
| Compatibility | Support modern web browsers. |

### 4.4 Hardware and Software Requirements

The application requires a standard computer or laptop and suitable software tools.

**Table 4.3: System Requirements**

| Category | Requirement |
|---|---|
| Operating System | Windows |
| Programming Language | Python |
| Backend Framework | Flask |
| Frontend | HTML, CSS, JavaScript |
| Data Storage | JSON File |
| Chart Library | Chart.js |
| Web Browser | Google Chrome or equivalent |
| Hardware | Desktop or Laptop |

### 4.5 Data Requirements

Library Studio uses a JSON file named library_data.json to store its records.

The stored information includes:

- Book details and availability.
- Member details.
- Issue dates and due dates.
- Fine-related information.
- Dashboard statistics.

The current implementation uses JSON-based storage and does not depend on a relational database.

**4.6 System Constraints**

The major constraints are:

1. The application uses JSON files instead of a dedicated database.
2. It is designed as a standalone web application.
3. The borrowing period is fixed at 14 days.
4. The overdue fine is fixed at ₹5 per day.
5. A modern web browser is required.

**4.7 Summary**

This chapter describes the functional requirements, quality requirements, system resources, and constraints of Library Studio. These requirements form the basis for system design and development.

CHAPTER 5

# SYSTEM FLOW

System flow represents the movement of information and the interaction between different components of an application. It explains how users interact with the system and how different operations are processed.

Library Studio uses different modules to manage books, members, book circulation, fine calculation, and dashboard information.

**5.1 Context Level Diagram (CLD)**

The Context Level Diagram represents Library Studio as a single system and shows its interaction with the Library Operator. The operator provides book details, member details, and circulation requests. The system processes the information and displays updated records, availability, fine details, and dashboard statistics.

**5.2 Data Flow Diagram (DFD) – Level 0**

DFD Level 0 represents the major processes of Library Studio and the movement of data between the operator, application modules, and JSON data storage.

The main processes include book management, member management, book circulation, fine calculation, and dashboard generation.

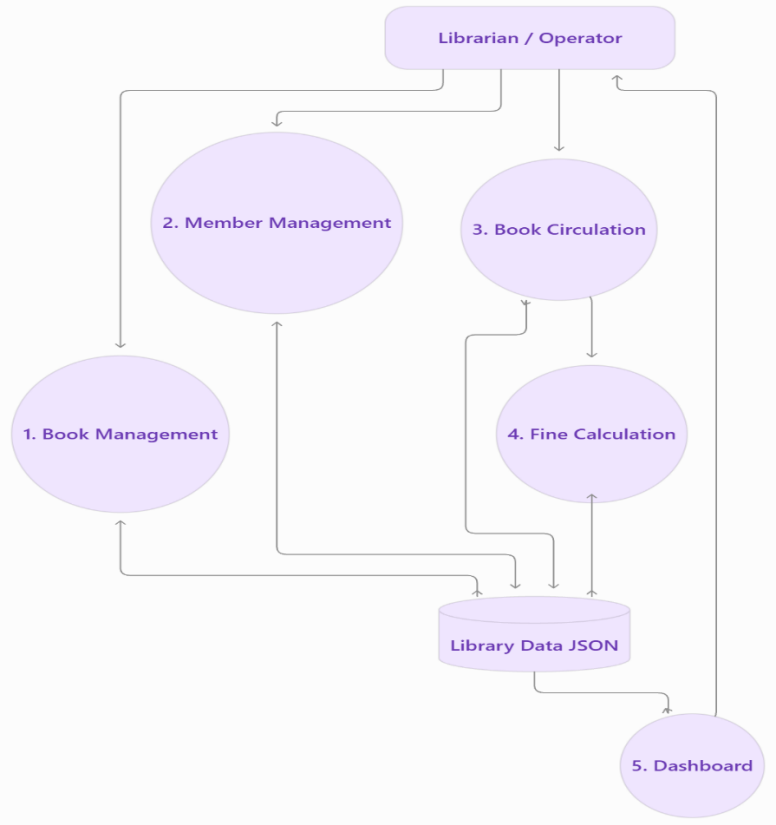



**Figure 5.1: Data Flow Diagram – Level 0**

### 5.3 Use Case Diagram

The Use Case Diagram represents the interaction between the Library Operator and the application. It shows the major operations, including managing books, registering members, issuing and returning books, calculating fines, and viewing dashboard statistics.

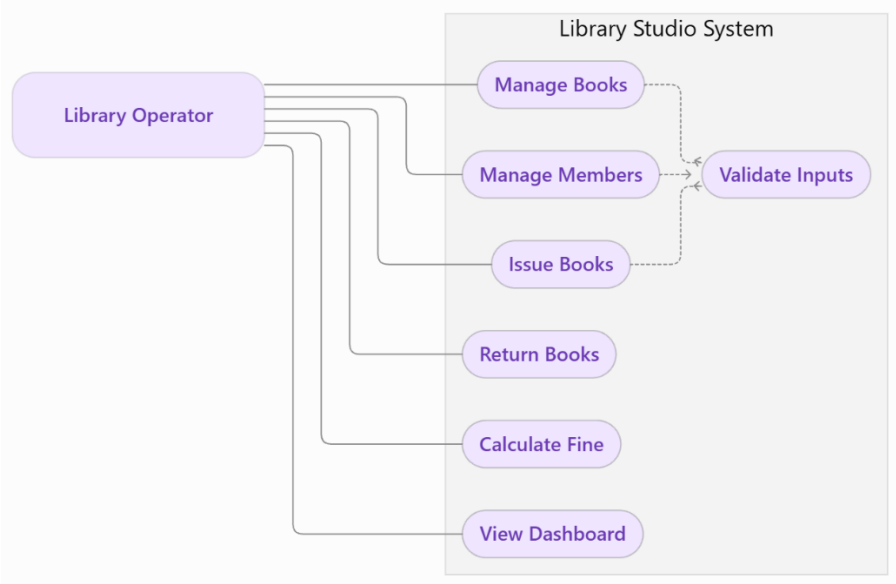



**Figure 5.2: Use Case Diagram**

### 5.4 Entity–Relationship Diagram (ERD)

The Entity–Relationship Diagram represents the logical data entities used in Library Studio. The main entities are Book, Member, and Issue Record. The Issue Record connects a book with a member and maintains issue-related information.

This is a conceptual data model. The current application stores records in JSON format and does not use a relational database.

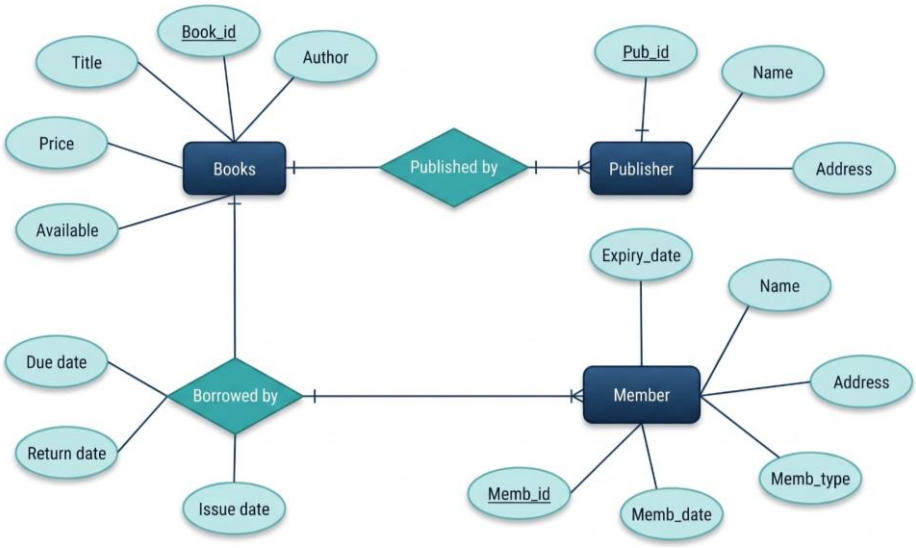



**Figure 5.3: Entity–Relationship Diagram**

### 5.5 Activity Diagram

The Activity Diagram represents the sequence of activities performed in the application. It includes user interaction, operation selection, input validation, data processing, record updating, and result display.

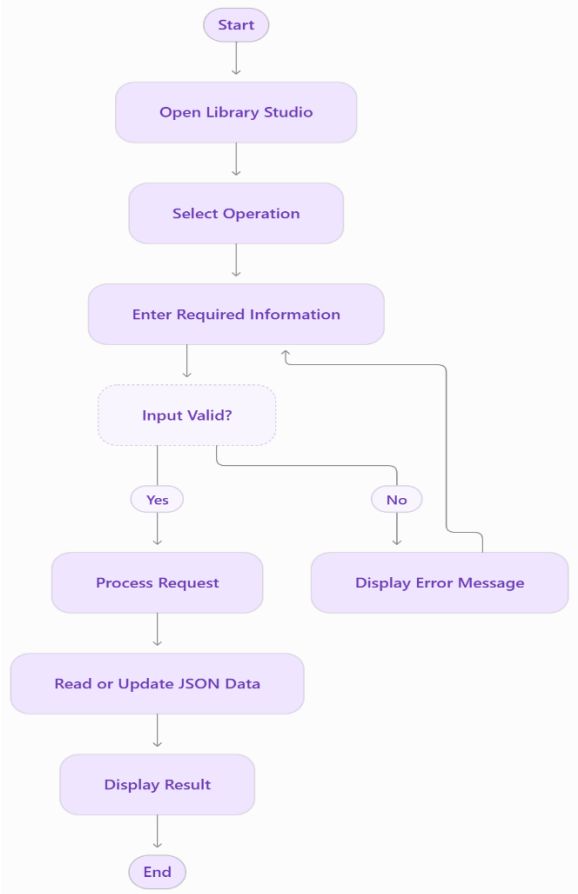



**Figure 5.4: Activity Diagram**

### 5.6 Sequence Diagram

The Sequence Diagram explains the interaction between the user interface, Flask application, application logic, and JSON data storage. It shows how a request is submitted, processed, and returned as a response.

### 5.7 Book Issue and Return Flowchart

The flowchart represents the logical steps involved in issuing and returning a book. During issuing, the system checks member details and book availability before recording the issue date and due date. During returning, it calculates any overdue fine and updates the book's availability.

### 5.8 Summary

This chapter explains the system flow of Library Studio through diagrams. These diagrams provide a clear understanding of user interaction, data movement, system operations, and book circulation.

CHAPTER 6

# PROPOSED DESIGN

## 6.1 Design Approach

Library Studio follows a modular design approach. The application is divided into different components to manage books, members, circulation, data storage, and dashboard operations.
The main design objectives are simplicity, usability, data consistency, and easy maintenance.

## 6.2 System Architecture Design

Library Studio follows a web-based architecture. The user interacts with the frontend interface, while the Flask backend processes requests and communicates with the JSON data file.
The major components are:

1. **Presentation Layer:** HTML, CSS, and JavaScript provide the user interface.
2. **Application Layer:** Python and Flask handle requests, validation, and library operations.
3. **Data Storage Layer:** The library_data.json file stores book, member, and circulation records.
4. **Visualization Layer:** Chart.js displays library statistics through dashboard charts.

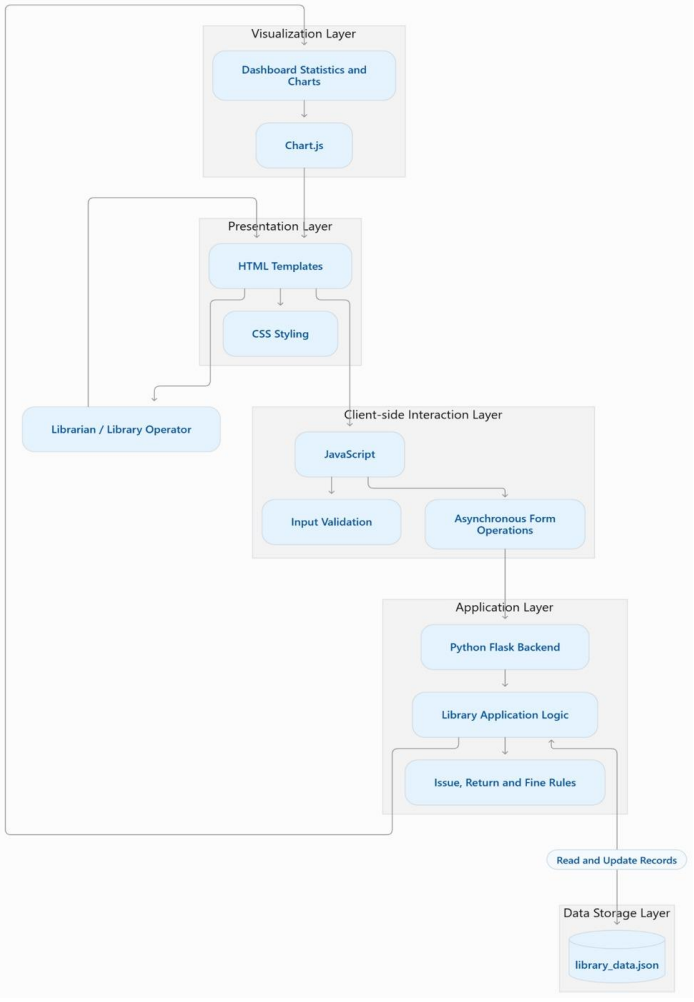



**Figure 6.1: System Architecture Diagram**

### 6.3 User Interface Design

The user interface is designed to provide easy access to the main library operations.

The major sections include:

1. **Overview Dashboard:** Displays total books, available books, issued books, registered members, and charts.
2. **Book Catalogue:** Provides facilities to add and manage book records.
3. **Member Management:** Allows registration and management of members.
4. **Circulation Management:** Supports book issuing, returning, due-date handling, and fine calculation.

Actual application screenshots should be inserted in Chapter 7 after running the project.

### 6.4 Data Storage Design

Library Studio uses a JSON file named library_data.json for persistent data storage. It maintains book, member, issue, return, and fine-related information.

**Table 6.1: JSON Data Storage Structure**

| Data Category | Information Maintained |
|---|---|
| Book | Book ID, title, author, availability |
| Member | Student ID and name |
| Issue | Book ID, member ID, issue date, due date |
| Return | Return-related status and details |
| Fine | Overdue days and fine amount |

The current implementation uses file-based JSON storage and does not use a relational database.

### 6.5 Design Considerations

The following considerations were followed during the design and development of the application:

1. **Simplicity:** Easy-to-understand interface.
2. **Modularity:** Separate functional components.
3. **Data Consistency:** Correct updating of book and member records.
4. **Input Validation:** Prevention of invalid entries.
5. **Record Protection:** Restrictions on deleting books or members with active issues.
6. **User Feedback:** Display of success messages, errors, and fine notifications.
7. **Maintainability:** Organized application structure.

### 6.6 Implementation Details

The application is implemented using Python and Flask for backend processing, HTML and CSS for the interface, and JavaScript for client-side interactions. JSON is used for storing application records, while Chart.js is used for dashboard visualization.

The main implementation files include:

- app.py – Flask application and backend operations.
- library_data.json – Stores library records.
- templates/index.html – Main web interface.
- static/style.css – Interface styling.

### 6.7 Summary

This chapter describes the design structure, architecture, user interface, data storage, and implementation technologies of Library Studio. The modular design supports organized development and future improvements.

CHAPTER 7

# EXPERIMENTAL RESULT

### 7.1 Overview of Experimental Setup

The experimental phase focuses on evaluating the main functionalities of Library Studio. The application is developed using Python and Flask, with HTML, CSS, JavaScript, and JSON-based data storage.

The major areas considered for testing include book management, member management, book issuing and returning, fine calculation, input validation, and dashboard statistics.

### 7.2 Functional Testing

Functional testing is used to check whether the main modules behave according to their specified requirements.

**Table 7.1: Consolidated System Testing**

| Sl. No. | Test Scenario | Expected Result |
|---|---|---|
| 1 | Add a book with valid details | Book record should be saved. |
| 2 | Enter invalid book details | Validation error should appear. |
| 3 | Register a valid member | Member record should be saved. |
| 4 | Delete a member with an active issue | Deletion should be prevented. |
| 5 | Issue an available book | Book should be issued successfully. |
| 6 | Issue an unavailable book | Issue operation should be rejected. |
| 7 | Return an issued book | Book should become available. |
| 8 | Return a book after 14 days | Fine should be calculated. |
| 9 | Add or remove a member | Dashboard count should update. |
| 10 | Issue or return a book | Availability statistics should update. |

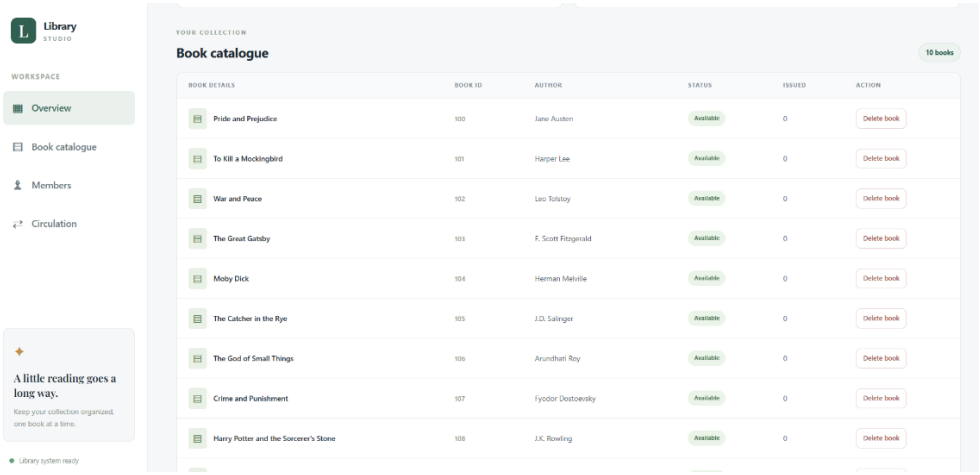



**Figure 7.2 – Book Catalogue Screen**

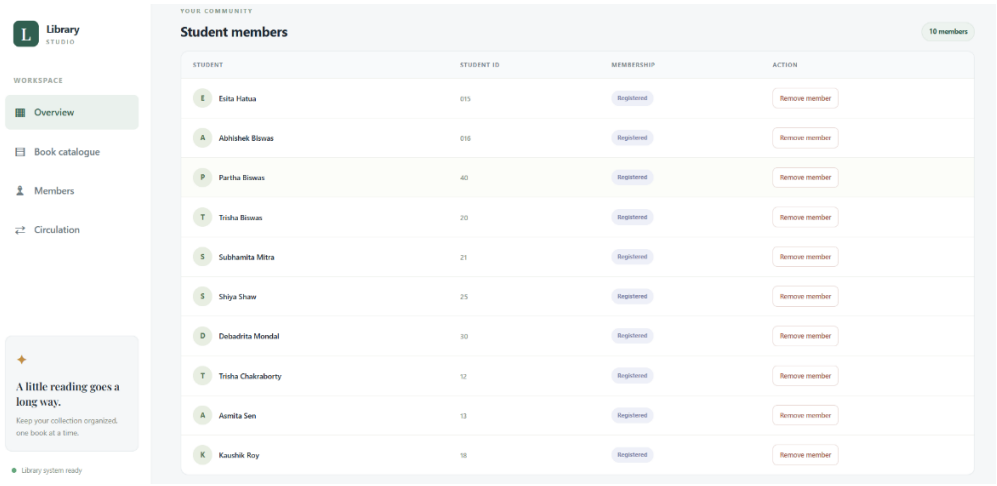



**Figure 7.3 – Member Management Screen**

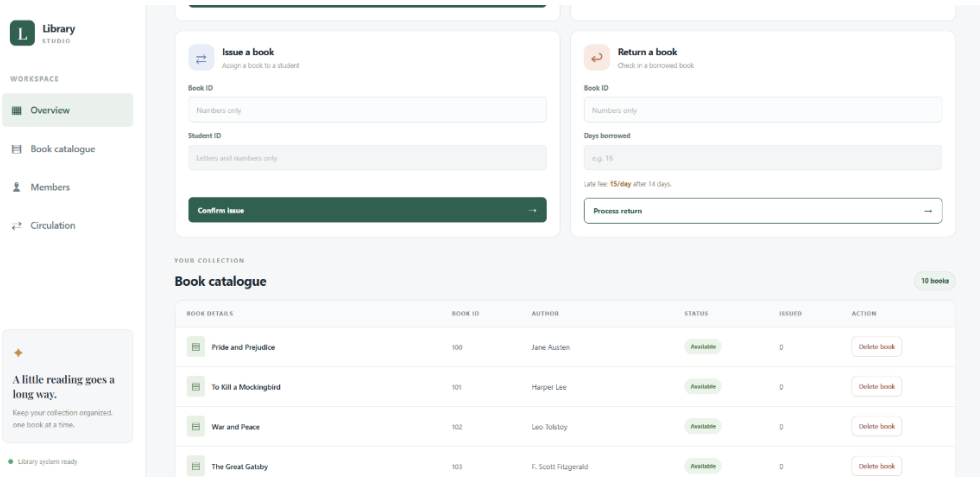



**Figure 7.4 – Book Issue and Return Screen**

### 7.3 Fine Calculation Results

Library Studio calculates overdue fines according to a fixed borrowing period of 14 days. A fine of ₹5 is charged for each additional day.

The calculation is:

**Fine Amount = Late Days × ₹5**

For example, if a book is returned 3 days late:

Fine Amount = 3 × ₹5 = ₹15

**Table 7.2: Fine Calculation Examples**

| Late Days | Fine Amount |
|---|---|
| 0 Days | ₹0 |
| 1 Day | ₹5 |
| 3 Days | ₹15 |
| 5 Days | ₹25 |

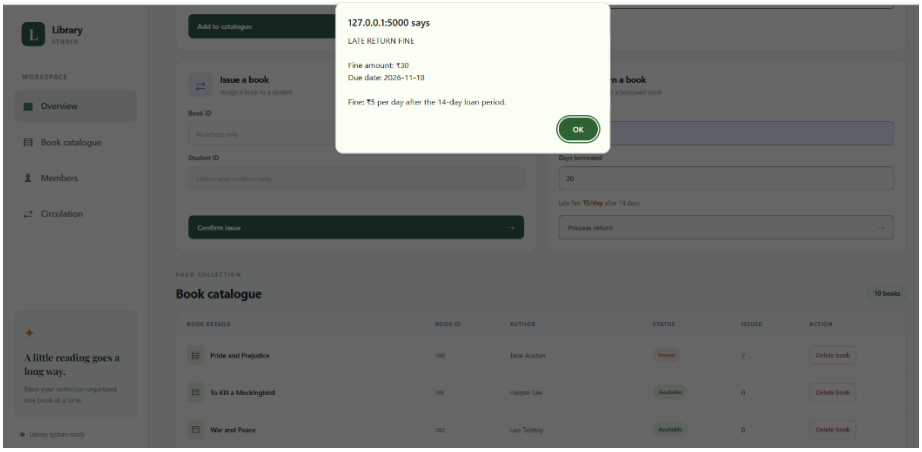
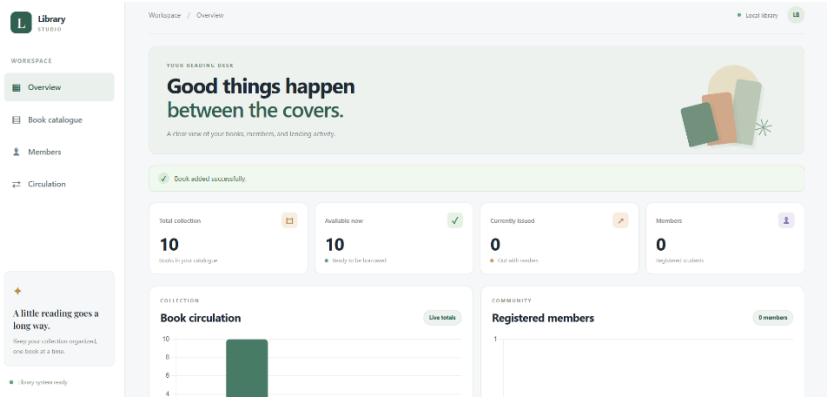



**Figure 7.5 – Fine Notification Popup**

## 7.4 Dashboard and Analytics Results

The dashboard provides a summarized view of library information. It displays total books, available books, issued books, and registered members.

Chart.js is used to present graphical information, including available-versus-issued books and member-related statistics.



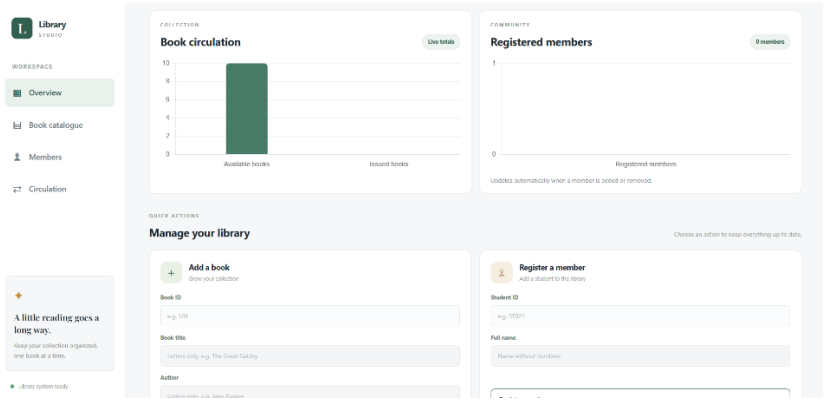



**Figure 7.1: Library Dashboard Screen**

### 7.5 Performance Observations

The application is designed to process routine operations such as adding books, registering members, issuing books, and returning books.

The main observation areas include:

1. Correct processing of form submissions.
2. Proper input validation.
3. Accurate updating of JSON records.
4. Correct dashboard statistics.
5. Clear user feedback after operations.

Actual response times and performance measurements should only be included after testing the application in the selected environment.

### 7.6 Summary

This chapter describes the testing areas and expected behaviour of Library Studio. It covers the major application modules, fine calculation, and dashboard functionality. Application screenshots provide visual evidence of the implemented interface.

CHAPTER 8

# LIMITATION AND FUTURE SCOPE

### 8.1 Limitations

Although the Library Studio system provides essential library management features, it has some limitations:

1. **JSON-Based Storage:** The system uses JSON files instead of a relational database, which may not be suitable for handling a very large amount of data.
2. **Single-Operator Usage:** The current system is mainly designed for operation by a librarian or library operator.
3. **Limited Security:** Advanced security features, such as role-based access control, are not included.
4. **Internet Dependency:** The web application requires a suitable environment to run and access its web interface.
5. **Basic Reporting:** The dashboard provides basic statistics and visual summaries rather than advanced analytical reports.

### 8.2 Future Scope

The system can be improved in several ways in the future:

1. **Database Integration:** MySQL or SQLite can be integrated to manage records more efficiently.
2. **User Authentication:** Login systems with different access levels can be introduced.
3. **Automated Notifications:** Email or SMS notifications can be added for due dates and overdue books.
4. **Advanced Reports:** More detailed reports can be generated for book circulation, membership and fines.
5. **Online Book Reservation:** Members may be allowed to reserve books through an online interface.
6. **Cloud Deployment:** The application can be hosted on a cloud platform for remote access.

### 8.3 Summary

This chapter discussed the limitations of the current Library Studio system and identified possible improvements. These enhancements can make the application more scalable, secure and useful for future library management requirements.

CHAPTER 9

# CONCLUSION

### 9.1 Conclusion

The **Library Studio: Library Management System** was developed to simplify and organize the basic activities of library management. The application provides features for book management, member management, book issuing, book returning and fine calculation.

The system was developed using Python and Flask for backend operations, HTML, CSS and JavaScript for the user interface, JSON files for data storage, and Chart.js for dashboard visualization.

The project helped demonstrate how programming concepts can be applied to solve practical problems. It also provided experience in web application development, data handling, system design and software testing.

Although the current system has certain limitations, it successfully demonstrates the basic workflow of a library management application. Future improvements can make it more scalable, secure and efficient.

### 9.2 Learning Outcomes

During the development of this project, the following concepts and skills were explored:

1. Understanding the software development process.
2. Developing web applications using Python and Flask.
3. Designing user interfaces using HTML, CSS and JavaScript.
4. Managing application data using JSON files.
5. Implementing book issue, return and fine calculation logic.
6. Understanding system design and functional testing.

### 9.3 Final Statement

In conclusion, Library Studio provides a simple and organized approach to managing essential library operations. The project meets its basic objectives and provides a foundation for further development of a more advanced library management system.

# REFERENCES


1. The following documentation and technical resources were referred to during the development and preparation of the Library Studio project report.
2. Python Software Foundation. *Python Documentation*. Available at: https://docs.python.org/3/
3. Pallets Projects. *Flask Documentation*. Available at: https://flask.palletsprojects.com/
4. Pallets Projects. *Jinja Documentation*. Available at: https://jinja.palletsprojects.com/
5. Mozilla Developer Network (MDN). *HTML Documentation*. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML
6. Mozilla Developer Network (MDN). *CSS Documentation*. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS
7. Mozilla Developer Network (MDN). *JavaScript Documentation*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript
8. Chart.js. *Chart.js Documentation*. Available at: https://www.chartjs.org/docs/latest/
9. Python Software Foundation. *JSON — JSON Encoder and Decoder*. Available at: https://docs.python.org/3/library/json.html
10. Library Studio Project Source Code. *Library Management System*, including app.py, library_data.json, templates/index.html, and static/style.css. Project source files.
11. **Date of Access:** 03 November 2026.